

Методичні вказівки: Цикли в Python

Оператор циклу `for` та функція `range()`

Оператори циклів використовують у випадках, коли в програмі необхідно багаторазово виконати певну послідовність команд. У мові Python основним циклом із заздалегідь відомою кількістю повторень є цикл `for`.

Синтаксис циклу for

```
for параметр in послідовність:
    блок_операторів
```

1. Генератор послідовностей `range()`

Найчастіше для формування послідовностей чисел у циклі `for` використовується вбудована функція `range()`. Вона може приймати від 1 до 3 аргументів:

- `range(stop)` — генерує числа від 0 до *stop* – 1 із кроком 1.
- `range(start, stop)` — генерує числа від *start* до *stop* – 1 із кроком 1.
- `range(start, stop, step)` — генерує числа від *start* до *stop* – 1 із кроком *step*.

Виклик `range()`	Згенерована послідовність	Примітка
`range(5)`	`0, 1, 2, 3, 4`	Від 0 до 4
`range(1, 6)`	`1, 2, 3, 4, 5`	Від 1 до 5
`range(2, 10, 2)`	`2, 4, 6, 8`	Парні числа від 2 до 8
`range(10, 0, -1)`	`10, 9, 8, 7, 6, 5, 4, 3, 2, 1`	Зворотний відлік

Таблиця 1: Приклади роботи функції `range()`

2. Базові алгоритми з циклом `for`

2.1. Обчислення суми послідовності

Знайдемо суму перших n натуральних чисел: $S = 1 + 2 + \cdots + n$.

```
n = int(input("Введіть n: "))
s = 0  # Початкове значення суми (нейтральний елемент відносно додавання)

for i in range(1, n + 1):
    s += i

print(f"Сума чисел від 1 до {n} дорівнює: {s}")
```

2.2. Обчислення факторіала *n*!

Факторіал числа *n* — це добуток усіх натуральних чисел від 1 до *n*: $n! = 1 \cdot 2 \cdot \dots \cdot n$.

```
n = int(input("Введіть n: "))
fact = 1  # Початкове значення добутку

for i in range(1, n + 1):
    fact *= i

print(f"{n}! = {fact}")
```

2.3. Табулювання функції

Програма обчислює значення функції $y = \sin(x)$ на відрізку $[a, b]$ з однаковим кроком h:

```
import math

a = float(input("Введіть початок відрізка a: "))
b = float(input("Введіть кінець відрізка b: "))
n = 10  # Кількість розбиттів
h = (b - a) / n

print(" x | sin(x)")
print("-------------")
for i in range(n + 1):
    x = a + i * h
    y = math.sin(x)
    print(f"{x:6.2f} | {y:6.4f}")
```

3. Аналіз дільників числа та перевірка на простоту

Математична довідка

Просте число — це натуральне число, більше за 1, яке має точно два різні натуральні дільники: 1 та саме це число. Якщо число n має дільник, відмінний від 1 та n, то найменший такий дільник не перевищує $\lfloor \sqrt{n} \rfloor$.

3.1. Виведення всіх дільників числа

Знайдемо та виведемо всі дільники числа n, а також обчислимо їхню кількість:

```
n = int(input("Введіть натуральне число n: "))

divisors = []
for i in range(1, n + 1):
    if n % i == 0:
        divisors.append(i)

print(f"Дільники числа {n}: {divisors}")
print(f"Кількість дільників: {len(divisors)}")
```

3.2. Оптимізована перевірка числа на простоту

Щоб перевірити, чи є число n простим, достатньо шукати дільники в діапазоні від 2 до $\lfloor \sqrt{n} \rfloor$:

```
import math
```

```
n = int(input("Введіть число n (n > 1): "))

if n <= 1:
    print("Число має бути більшим за 1")
else:
    is_prime = True
    limit = int(math.isqrt(n))  # Цілочисельний квадратний корінь

    for i in range(2, limit + 1):
        if n % i == 0:
            is_prime = False
            break  # Достроково перериваємо цикл, якщо знайшли дільник

    if is_prime:
        print(f"Число {n} є простим!")
    else:
        print(f"Число {n} є складеним.")
```

3.3. Комплексний приклад: дільники та статус простоти

Програма виводить усі дільники числа та визначає, чи є воно простим:

```
import math

n = int(input("Введіть число n: "))

if n <= 0:
    print("Введіть додатне натуральне число!")
else:
    divisors = []
    # Знаходимо всі дільники
    for i in range(1, n + 1):
        if n % i == 0:
            divisors.append(i)

    print(f"Усі дільники числа {n}: {divisors}")

    # Перевіряємо на простоту
    if len(divisors) == 2:
        print(f"Число {n} є ПРОСТИМ.")
    elif n == 1:
        print("Число 1 не є ні простим, ні складеним.")
    else:
        print(f"Число {n} є СКЛАДЕНИМ (має {len(divisors)} дільників).")
```

4. Обчислення нескінченних математичних рядів

4.1. Сума геометричного ряду

Обчислимо суму $S = \sum_{n=0}^{N} x^n = 1 + x + x^2 + \cdots + x^N$ ($|x| < 1$).

Оптимізуємо обчислення: замість піднесення до степеня `x**i` на кожному кроці будемо множити попереднє значення на x:

```
x = float(input("Введіть x (|x| < 1): "))
n = int(input("Введіть кількість членів N: "))

s = 0.0
term = 1.0  # Перший член x^0 = 1

for i in range(n):
    s += term
    term *= x  # Наступний член ряду отримаємо множенням на x

print(f"Сума ряду: {s:.6f}")
print(f"Точна сума 1/(1-x): {1 / (1 - x):.6f}")
```

4.2. Обчислення експоненти e^x через ряд Тейлора

Формула Тейлора: $e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$

```
import math

x = float(input("Введіть x: "))
n = int(input("Кількість членів ряду: "))

s = 0.0
term = 1.0  # Перший доданок (x^0 / 0! = 1)

for i in range(1, n + 1):
    s += term
    term *= x / i  # Рекурентне співвідношення для наступного доданка

print(f"Наближене значення e^{x} = {s:.8f}")
print(f"Точне значення (math.exp): {math.exp(x):.8f}")
```

5. Управління циклом: `break`, `continue` та `else`

- `break` — достроково припиняє виконання циклу.
- `continue` — пропускає залишок тіла циклу та переходить до наступної ітерації.
- `else` — блок коду, який виконується після завершення циклу, якщо вихід з нього **не** відбувся за допомогою `break`.

```
# Пошук першого від'ємного числа в послідовності
for i in range(5):
    val = int(input(f"Введіть число #{i+1}: "))
    if val < 0:
        print(f"Знайдено перше від'ємне число: {val}")
        break
else:
    print("Усі введені числа є невід'ємними.")
```

6. Оператори інкременту та особливості `float`

У Python відсутні оператори `++` та `--`. Замість них використовуються комбіновані оператори присвоєння:

- `x += 1` замість `x = x + 1`
- `x -= 1` замість `x = x - 1`
- `x *= 2` замість `x = x * 2`

Обережно з дійсними числами (float)!

Числа з плаваючою крапкою зберігаються в пам'яті комп'ютера у двійковій системі з обмеженою точністю (53 біти). Через це обчислення можуть містити незначну похибку:

```
>>> 0.1 + 0.1 + 0.1
0.30000000000000004
```

Для порівняння речових чисел завжди використовуйте функцію `math.isclose(a, b)` або порівняння з точністю ε: `abs(a - b) < 1e-9`.

7. Завдання для лабораторної роботи

1. **Аналіз дільників:** Користувач вводить натуральне число N. Знайти всі його парні дільники та їх суму.
2. **Пошук простих чисел:** Надрукувати всі прості числа на проміжку від A до B ($1 < A < B$).
3. **Обчислення суми ряду:** Обчислити суму перших n членів ряду з заданою точністю ε:

$$S = \sum_{k=1}^{n} \frac{(-1)^{k+1}}{k^2} = 1 - \frac{1}{4} + \frac{1}{9} - \frac{1}{16} + \dots$$

4. **Найменший спільний дільник:** Для двох введених чисел A та B знайти їх найбільший спільний дільник (НСД) за допомогою циклу `for`.